

RAMAI

04 - Technical Architecture v1.0

Architettura tecnica dell'AI Business Operating Assistant per PMI

Scopo. Definire una base tecnica modulare, sicura e scalabile per l'MVP e per l'evoluzione a 5 anni, progettata per essere sviluppata in gran parte da Codex attraverso specifiche, test automatici e task indipendenti.

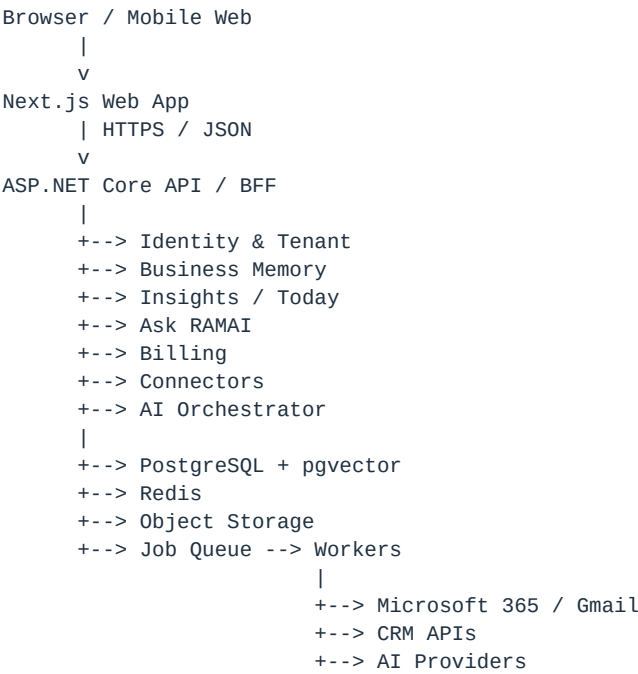
Decisione	Scelta v1.0
Approccio	Modular Monolith + worker asincroni
Backend	.NET 10 / ASP.NET Core
Frontend	Next.js / React / TypeScript
Database	PostgreSQL + pgvector
Cache / Queue	Redis + coda gestita
AI	Provider abstraction + OpenAI come provider primario
Identity	OIDC/OAuth 2.1; autenticazione locale + social/enterprise in futuro
Hosting	Cloud EU region, container-first
Tenancy	Shared database, row-level tenant isolation per MVP

Stato documento: baseline architetturale. Ogni modifica strutturale dovrà essere documentata tramite ADR (Architecture Decision Record).

1. Principi architetturali

- Semplicità prima della distribuzione: evitare microservizi prematuri; separare i moduli nel codice e nei contratti.
- Multi-tenant by design: ogni record applicativo deve essere riconducibile a un TenantId.
- API first: la UI è un client delle API applicative; le integrazioni esterne passano da adapter dedicati.
- Asincronia per i carichi pesanti: import email, embedding, analisi AI, sync CRM e briefing non devono bloccare richieste web.
- AI non deterministica ma controllata: output strutturati, validazione schema, citazione delle evidenze e fallback.
- Privacy e sicurezza come requisiti di base, non feature successive.
- Observability first: log, metriche, tracing e costo AI devono essere disponibili dall'MVP.
- Codex-friendly: repository leggibile, moduli piccoli, test chiari, convenzioni rigide e Definition of Done automatizzabile.

2. Vista logica di alto livello



La prima release è un **modular monolith**: un singolo deploy logico per l'API, con moduli interni isolati e worker separati per i job. Questo riduce complessità operativa e rende comunque possibile estrarre servizi futuri senza riscrivere il dominio.

3. Stack tecnologico raccomandato

Area	Tecnologia	Motivo
Frontend	Next.js + React + TypeScript	SSR/SPA ibrida, ecosistema maturo, UI veloce da generare/testare con C
Backend	ASP.NET Core .NET 10	Prestazioni, tipizzazione, security tooling, background processing, esperienze
ORM	EF Core	Migrazioni, query tipizzate, interceptors tenant, auditing
DB	PostgreSQL	Affidabile, economico, JSONB, full text e pgvector
Vector	pgvector	Evita un database vettoriale separato nell'MVP

Area	Tecnologia	Motivo
Cache	Redis	Session/cache, rate limit, distributed locks, short-lived state
Queue	Azure Service Bus / AWS SQS / JMS	Idempotente, retry, dead-letter queue
Storage	Blob/S3 compatibile	Allegati, export, artefatti temporanei
Observability	OpenTelemetry + provider gestione	Tracing end-to-end e metriche
CI/CD	GitHub Actions	Automazione semplice, test/security/deploy

4. Multi-tenancy

4.1 Modello MVP

Per i primi anni è consigliato un **database condiviso con schema condiviso**, dove tutte le tabelle di dominio contengono TenantId. È il miglior compromesso tra costi, semplicità operativa e scalabilità fino a migliaia di clienti PMI.

- TenantId obbligatorio su tutte le entità tenant-scoped.
- Global query filters EF Core per filtrare automaticamente il tenant.
- TenantContext risolto dal token autenticato, mai da parametri liberi del client.
- Indici composti TenantId + chiave di ricerca sulle tabelle ad alto volume.
- Test automatici specifici contro data leakage tra tenant.

4.2 Evoluzione futura

Per clienti enterprise o requisiti regolatori si potrà introdurre un modello ibrido: tenant standard nel database condiviso; tenant premium su database dedicato. Il dominio applicativo non deve dipendere dalla topologia fisica del database.

5. Moduli applicativi

Modulo	Responsabilità
Identity & Access	utenti, login, ruoli, sessioni, MFA futuro
Tenant Management	azienda, piano, membership, limiti, configurazioni
Connector Hub	OAuth, token vault, sync state, mapping provider
Communication	email, thread, partecipanti, allegati e metadati
Business Memory	contatti, aziende, relazioni, eventi, timeline, facts
Insight Engine	regole + AI, scoring, motivazioni, deduplica insight
Today	prioritizzazione delle azioni giornaliere
Ask RAMAI	query contestuali, retrieval, tool calling read-only nell'MVP
Drafting	preparazione bozze email e azioni suggerite
Billing	piani, usage, entitlement, pagamento e webhook
Notification	daily briefing, notifiche prodotto
Audit & Compliance	audit log, consensi, export/delete, retention

6. Struttura repository

```
/ramai
  /apps
    /web          # Next.js
    /api          # ASP.NET Core HTTP API
    /worker       # background jobs
  /src
    /Ramai.Domain
    /Ramai.Application
    /Ramai.Infrastructure
```

```
/Ramai.Modules.Identity
/Ramai.Modules.Tenants
/Ramai.Modules.Connectors
/Ramai.Modules.Memory
/Ramai.Modules.Insights
/Ramai.Modules.Assistant
/Ramai.Modules.Billing
/tests
  /Unit
  /Integration
  /Architecture
  /E2E
/docs
  /adr
  /api
  /runbooks
/infra
  /docker
  /iac
```

Regola: nessun modulo può accedere direttamente alle tabelle interne di un altro modulo. La comunicazione avviene attraverso contratti applicativi o eventi interni.

7. Business Memory: fondazione dati

La Business Memory è la proprietà tecnica centrale di RAMAI. Deve rappresentare fatti aziendali con provenienza verificabile e non una semplice memoria conversazionale del modello.

Entità	Campi chiave / note
Organization	id, tenant, nome, dominio, stato cliente/lead
Person	nome, email, telefono, organization, ruoli
Conversation	provider, thread id, subject, timestamps
Message	conversation, sender, recipients, body normalized, source id
Opportunity	valore, stage, owner, last activity, source
Task	tipo, due date, status, confidence, source
Event	meeting/call/activity con timestamp e partecipanti
Fact	subject, predicate, value, validity, confidence, provenance
Insight	tipo, severity, score, rationale, evidence links, lifecycle
Embedding	entity type/id, chunk, vector, model/version

7.1 Provenance

Ogni fatto generato dall'AI deve conservare **source type, source id, timestamp e confidence**. Un insight mostrato all'utente deve poter risalire alle email, eventi o record CRM che lo giustificano.

7.2 Event model

Le integrazioni normalizzano dati eterogenei in eventi interni: EmailReceived, EmailSent, MeetingScheduled, OpportunityChanged, TaskCompleted, ContactUpdated. L'Insight Engine reagisce a questi eventi senza conoscere direttamente i provider esterni.

8. Connector architecture

External Provider -> Provider Adapter -> Normalizer -> Internal Event / Entity -> Business Memory

- Ogni provider implementa un contratto comune: Connect, RefreshToken, SyncIncremental, FetchObject, Disconnect.
- I token OAuth devono essere cifrati separatamente dai dati applicativi e mai esposti al frontend.
- La sincronizzazione deve essere incrementale con cursor/delta token quando disponibile.
- Idempotenza obbligatoria: lo stesso oggetto remoto importato due volte non deve generare duplicati.
- Rate limit e backoff specifici per provider.
- Dead-letter queue per elementi non elaborabili con reprocessing manuale o automatico.

8.1 Priorità connettori

Fase	Connettori
MVP-A	Microsoft 365 Outlook Mail + Calendar
MVP-B	Gmail + Google Calendar

Fase	Connettori
Beta+	HubSpot / Pipedrive
Successivo	Salesforce, Teams, Slack, WhatsApp Business, VoIP

9. AI Orchestration Layer

RAMAI non deve chiamare direttamente un modello da controller o UI. Tutto passa da un livello di orchestrazione che centralizza prompt, modelli, policy, costi, logging e validazione.

```
Feature -> AI Orchestrator -> Policy / Router -> Provider Adapter -> Model
|
+--> Structured Output Validator
+--> Token & Cost Meter
+--> Safety / PII policy
+--> Prompt Registry
+--> Evaluation hooks
```

9.1 Routing dei modelli

Task	Classe modello
classificazione semplice	economico / veloce
estrazione strutturata	economico o medio con JSON schema
sintesi thread	medio
reasoning su priorità	medio/alto solo quando necessario
Ask RAMAI complesso	routing dinamico con budget per piano

9.2 Regole non negoziabili

- Output strutturati con schema JSON per estrazioni e insight.
- Nessun fatto aziendale deve essere accettato come vero senza evidenza o source reference.
- Prompt versionati nel repository o in registry con version id persistito nel log.
- AIUsage registra tenant, feature, modello, token input/output, costo stimato, latency e risultato.
- Fallback tra modelli/provider per errori transitori, non per mascherare errori logici.
- Cache semantica solo dove il rischio di dati obsoleti è controllabile.

10. Pipeline di analisi email

1. Sync provider
2. Normalize message/thread
3. Deduplicate
4. Extract contacts + organizations
5. Lightweight classification
6. Detect commitments / questions / opportunity signals
7. Update Business Memory
8. Generate candidate insights
9. Score + deduplicate insights
10. Publish to Today / Briefing

Le fasi 1-9 sono asincrone. La UI deve mostrare progressivamente lo stato di importazione e rendere disponibili i primi insight appena pronti, senza attendere l'intera casella.

11. Insight scoring

Un insight non deve essere mostrato solo perché il modello lo considera interessante. Il punteggio deve combinare fattori deterministici e AI.

$PriorityScore = Urgency * BusinessValue * Confidence * Freshness * UserRelevance$

- Urgency: tempo trascorso, scadenze e SLA.

- BusinessValue: valore opportunità/cliente quando noto.
- Confidence: qualità delle evidenze e concordanza delle fonti.
- Freshness: penalizza segnali vecchi già superati.
- UserRelevance: ownership, ruolo e preferenze apprese.

12. API design

API versionate sotto **/api/v1**. REST è sufficiente per l'MVP; eventi e job gestiscono l'asincronia. Le API pubbliche future potranno riutilizzare gli stessi application services.

Area	Endpoint esempio
Session	GET /me
Tenants	GET /tenant, PATCH /tenant/settings
Connections	POST /connections/microsoft/start, DELETE /connections/{id}
Today	GET /today
Insights	GET /insights, GET /insights/{id}, POST /insights/{id}/dismiss
Memory	GET /organizations/{id}, GET /people/{id}/timeline
Assistant	POST /assistant/query
Drafts	POST /drafts/email
Usage	GET /usage
Billing	POST /billing/checkout, POST /webhooks/billing

12.1 API conventions

- ProblemDetails standard per gli errori.
- CorrelationId su ogni richiesta.
- Idempotency-Key per operazioni sensibili e webhook.
- Pagination cursor-based per collezioni ad alto volume.
- ETag/versioning ottimistico dove modifiche concorrenti sono possibili.
- Rate limiting per tenant e per endpoint AI.

13. Background processing

I worker gestiscono attività lente o soggette a retry. I job devono essere idempotenti, tracciabili e riprocessabili.

Queue/Job	Scopo
connector.sync	sincronizzazione provider
message.analyze	classificazione ed estrazione
memory.embed	embedding e indicizzazione
insight.generate	creazione candidati insight
insight.rescore	ricalcolo priorità
briefing.daily	creazione briefing tenant/user
retention.cleanup	cancellazione dati secondo policy

Ogni job salva JobRun: stato, tentativi, durata, errore sintetico, correlation id e tenant. Dopo N retry falliti passa in dead-letter queue.

14. Sicurezza applicativa

- OIDC/OAuth 2.1, cookie HttpOnly/Secure per web app o token BFF; evitare token sensibili nel browser.
- MFA obbligatoria per amministratori appena il prodotto entra in beta pubblica.
- RBAC iniziale: Owner, Admin, Member; autorizzazione sempre verificata server-side.
- Encryption at rest e in transit; segreti in managed secret store.
- Token OAuth provider cifrati con chiavi gestite e rotazione.
- OWASP ASVS come checklist; SAST, dependency scan e secret scanning in CI.
- Rate limit, anti-abuse e quota AI per prevenire costi incontrollati.
- Audit delle azioni sensibili: connessioni, export, delete, billing, autorizzazioni, azioni AI approvate.

15. Privacy, retention e isolamento dati

I dati email e CRM sono altamente sensibili. RAMAI deve minimizzare ciò che conserva e consentire politiche di retention configurabili.

- Conservare body completi solo se necessari al servizio e secondo policy trasparente.
- Separare metadati, contenuti e embedding per consentire cancellazione completa.
- La cancellazione di una sorgente deve invalidare insight/facts derivati o marcarli come orphaned prima del purge.
- Export account e delete account devono essere job asincroni auditati.
- Embeddings sono dati derivati: devono essere cancellati insieme alla fonte.
- No training dei modelli sui dati cliente salvo scelta esplicita e contrattuale del cliente.

16. Observability e FinOps AI

Segnale	Esempi
Logs	errori API, connector failures, auth events
Metrics	request latency, job latency, queue depth, sync lag
AI metrics	tokens, costo, latency, schema failures, retry rate
Product metrics	activation, first insight, daily briefing open, free-to-paid
Tracing	web -> API -> job -> provider -> AI call
Business health	tenant con sync bloccato, provider token expired

Ogni chiamata AI deve produrre un costo stimato e contribuire a un budget mensile per tenant. L'entitlement del piano deve poter imporre hard/soft limits senza modifica del codice della feature.

17. Deploy e ambienti

17.1 Ambienti

Ambiente	Uso
local	Docker Compose, provider sandbox/mocks
dev	integrazione continua e test team
staging	configurazione quasi-prod, beta pre-release
production	EU region, backup e monitoring completi

17.2 Container-first

Web, API e Worker vengono containerizzati. Il runtime può essere Azure Container Apps, AWS ECS/Fargate o equivalente. Kubernetes non è necessario nell'MVP e va introdotto solo se i requisiti operativi lo giustificano.

17.3 Infrastructure as Code

Database, queue, Redis, storage, secret store, DNS, monitoring e runtime devono essere descritti con Terraform/Bicep o equivalente. Nessuna risorsa critica deve esistere solo tramite configurazione manuale del portale cloud.

18. CI/CD e qualità

Pull Request

- > format/lint
- > unit tests
- > architecture tests
- > integration tests
- > SAST + dependency scan + secret scan
- > build containers
- > deploy dev
- > smoke tests
- > manual/automated promote staging
- > production

- Branch principale sempre deployabile.
- Feature flags per funzioni incomplete o sperimentali.
- Migrazioni database backward compatible quando possibile.
- Nessuna feature AI senza evaluation dataset minimo.
- Nessun connector senza test con mock/replay di payload reali anonimizzati.

19. Strategia di testing

Livello	Focus
Unit	dominio, scoring, mapping, entitlement
Architecture	dipendenze tra moduli e divieti di accesso
Integration	PostgreSQL, queue, Redis, provider adapters
Contract	payload provider e webhook billing
AI eval	estrazione, hallucination rate, ranking insight
E2E	signup -> connect -> sync -> insight -> Ask -> draft
Security	tenant isolation, authz, IDOR, token handling
Load	sync paralleli, queue depth, Today dashboard

19.1 Test di isolamento tenant

Devono esistere test automatici che tentano esplicitamente di accedere a record appartenenti a un altro tenant tramite ID validi. Questi test sono bloccanti in CI.

20. Scalabilità: quando estrarre servizi

Non utilizzare il numero di clienti come unico trigger. Un modulo viene estratto dal monolite quando ha almeno uno dei seguenti requisiti: scaling indipendente significativo, frequenza di deploy diversa, failure isolation necessaria, tecnologia incompatibile o ownership separata.

Possibile servizio futuro	Trigger
Connector Service	molti provider / sync ad alto volume
AI Processing Service	carico GPU/provider orchestration o forte scaling
Notification Service	volumi multicanale importanti

Possibile servizio futuro	Trigger
Search/Memory Service	necessità di storage/search indipendenti
Billing Service	complessità contrattuale multi-country

21. Capacità target e performance MVP

Indicatore	Target iniziale
API p95 non-AI	< 500 ms
Today dashboard p95	< 1.5 s con cache/aggregati
Ask RAMAI first response	< 5 s target, streaming dove utile
Primo insight onboarding	< 5 minuti dal consenso provider
Disponibilità beta pubblica	99.5%
RPO database	<= 15 minuti
RTO iniziale	<= 4 ore
Tenant supportati senza redesign	1.000+

22. Architecture Decision Records iniziali

ADR	Decisione
ADR-001	Modular Monolith per MVP
ADR-002	PostgreSQL + pgvector come datastore primario
ADR-003	Shared-schema multi-tenancy con TenantId
ADR-004	Asynchronous workers per sync e AI
ADR-005	AI Provider Abstraction Layer
ADR-006	Microsoft 365 primo connector
ADR-007	Container-first cloud deployment
ADR-008	No autonomous write actions nell'MVP

23. Sequenza tecnica di implementazione

Sprint tecnico	Deliverable
0	repository, conventions, CI, local stack, ADR
1	Identity + Tenant + DB baseline
2	Connector framework + Microsoft OAuth
3	Mail sync + normalization + worker
4	Business Memory core + provenance
5	AI Orchestrator + extraction pipeline
6	Insight Engine + Today API
7	Ask RAMAI retrieval + evidence
8	Drafting + Daily Briefing

Sprint tecnico	Deliverable
9	Billing/entitlements + usage limits
10	security hardening, observability, beta readiness

24. Definition of Architecture Ready

- Repository e module boundaries definite e testate.
- Modello tenant e authorization documentati.
- Data model Business Memory v1 definito con provenance.
- Contratto connector e primo provider definito.
- Contratto AI Orchestrator definito con structured output e cost metering.
- Queue/job conventions, retry e idempotency definiti.
- Pipeline CI/CD e ambienti stabiliti.
- Logging/tracing/metriche minime definite.
- Security checklist e threat model MVP completati.
- I task di Sprint 0-2 possono essere assegnati a Codex senza decisioni architetturali implicite.

Decisione conclusiva. RAMAI deve nascere come un prodotto semplice da operare ma rigoroso nella separazione dei moduli. La complessità va spostata nei contratti, nei test e nell'osservabilità, non nella distribuzione prematura in microservizi. Questa architettura permette di validare rapidamente l'MVP e offre una traiettoria credibile verso centinaia o migliaia di PMI senza replatforming immediato.